



VIT[®]
BHOPAL
www.vitbhopal.ac.in

Python Programming

Module 1

Content from Syllabus Unit 1

Unit No	Unit Content
1	A Brief History of Python, Different Versions, Python 2 vs Python 3, Installing Python, Environment Variables, Executing Python from the Command Line, Editing Python Files, Basic Python Syntax, String Values, String Operators, Numeric Data Types Conversions, Simple Input and Output, Language components - Control Flow structures and Syntax - Relational Operators - Logical Operators - Bit Wise Operators, Python for Windows

Python History and Versions

- Python laid its foundation in the late 1980s.
- The implementation of Python was started in December 1989 by **Guido Van Rossum** at CWI in Netherland.
- In February 1991, **Guido Van Rossum** published the code (labeled version 0.9.0) to alt.sources.
- In 1994, Python 1.0 was released with new features like lambda, map, filter, and reduce.
- Python 2.0 added new features such as list comprehensions, garbage collection systems.
- On December 3, 2008, Python 3.0 (also called "Py3K") was released. It was designed to rectify the fundamental flaw of the language.
- *ABC programming language* is said to be the predecessor of Python language, which was capable of Exception Handling and interfacing with the Amoeba Operating System.
- The following programming languages influence Python:
 - ABC language.
 - Modula-3

Why the Name Python?

There is a fact behind choosing the name Python

. **Guido van Rossum** was reading the script of a popular BBC comedy series "**Monty Python's Flying Circus**". It was late on-air 1970s.

Van Rossum wanted to select a name which unique, sort, and little-bit mysterious. So he decided to select naming Python after the "**Monty Python's Flying Circus**" for their newly created programming language.

The comedy series was creative and well random. It talks about everything. Thus it is slow and unpredictable, which made it very interesting.

Python is also versatile and widely used in every technical field, such as Machine Learning
, Artificial Intelligence
, Web Development, Mobile Application
, Desktop Application, Scientific Calculation, etc.

Python Version List

- Python programming language is being updated regularly with new features and supports. There are lots of update in Python versions, started from 1994 to current release.
- A list of Python versions with its released date is given below.

Python Version	Released Date
Python 1.0	January 1994
Python 1.5	December 31, 1997
Python 1.6	September 5, 2000
Python 2.0	October 16, 2000
Python 2.1	April 17, 2001
Python 2.2	December 21, 2001
Python 2.3	July 29, 2003
Python 2.4	November 30, 2004
Python 2.5	September 19, 2006
Python 2.6	October 1, 2008
Python 2.7	July 3, 2010
Python 3.0	December 3, 2008
Python 3.1	June 27, 2009

Python 3.1	June 27, 2009
Python 3.2	February 20, 2011
Python 3.3	September 29, 2012
Python 3.4	March 16, 2014
Python 3.5	September 13, 2015
Python 3.6	December 23, 2016
Python 3.7	June 27, 2018
Python 3.8	October 14, 2019

Tips to Keep in Mind While Learning Python

- The most common question asked by the beginners - "**What is the best way to learn Python**"? It is the initial and relevant question because first step in learning any programming language is to know how to learn.
- The proper way of learning will help us to learn fast and become a good Python developer.
- In this section, we will discuss various tips that we should keep in mind while learning Python.

1. Make it Clear Why We Want to Learn

The goal should be clear before learning the Python. Python is an easy, a vast language as well. It includes numbers of libraries, modules, in-built functions and data structures. If the goal is unclear then it will be a boring and monotonous journey of learning Python. Without any clear goal, you perhaps won't make it done.

So, first figure out the motivation behind learning, which can anything be such as knowing something new, develop projects using Python, switch to Python, etc. Below are the general areas where Python is widely used. Pick any of them.

- Data Analysis and Processing
- Artificial Intelligence
- Games
- Hardware/Sensor/Robots
- Desktop Applications

Choose any one or two areas according to your interest and start the journey towards learning Python.

- 2. Learn the Basic Syntax
- 3. Write Code by Own
- 4. Keep Practicing
- 5. Make Notes as Needed
- 6. Discuss Concepts with Other
- 7. Do small Projects
- 8. Teach Others

9. Explore Libraries and Frameworks

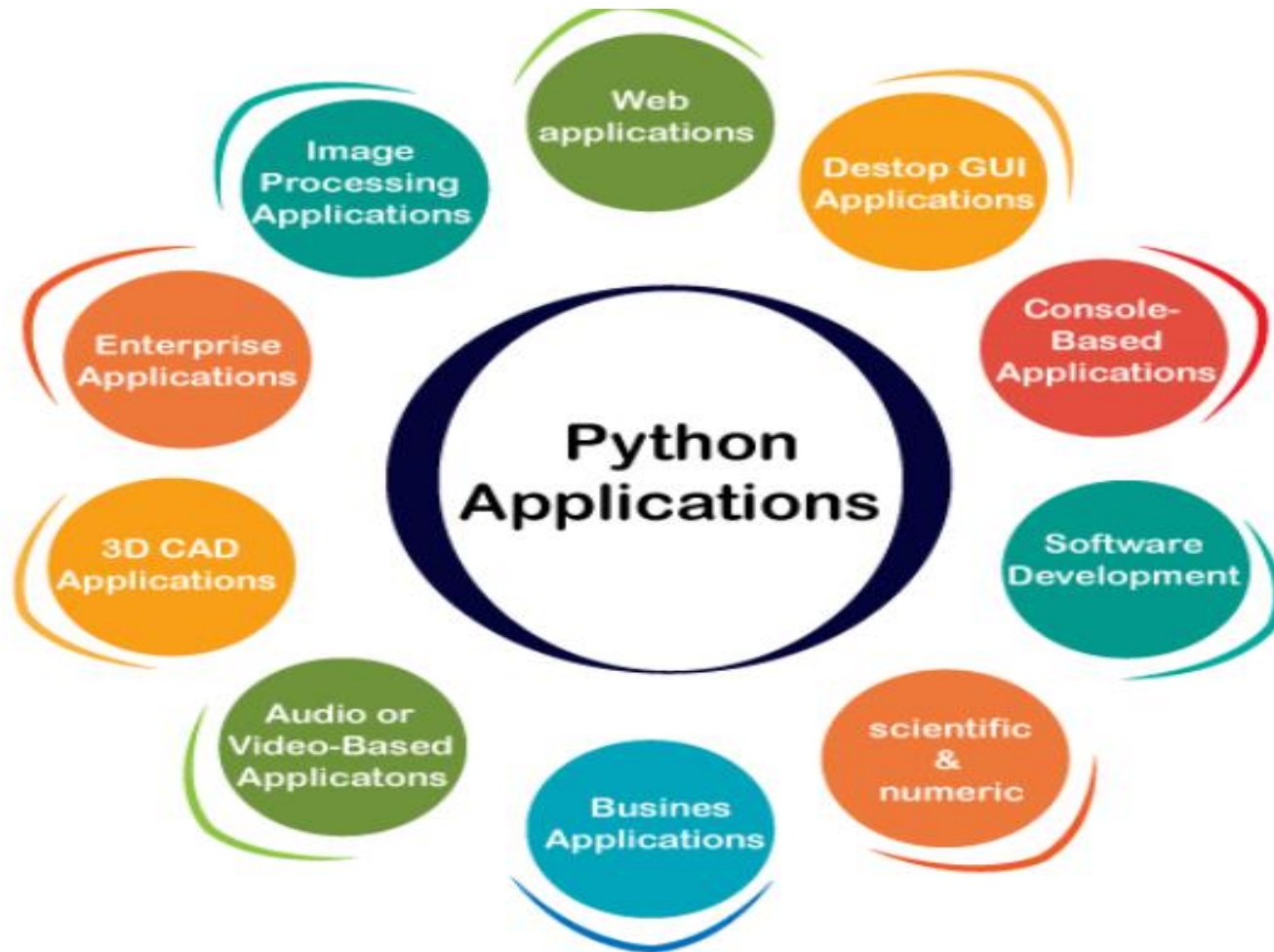
Python consists of vast libraries and various frameworks. After getting familiar with Python's basic concepts, the next step is to explore the Python libraries. Libraries are essential to work with the domain specific projects. In the following section, we describe the brief introduction of the main libraries.

- **TensorFlow**
 - It is an artificial intelligence library which allows us to create large scale AI based projects.
- **Django**
 - It is an open source framework that allows us to develop web applications. It is easy, flexible, and simple to manage.
- **Flask**
 - It is also an open source web framework. It is used to develop lightweight web applications.
- **Pandas**
 - It is a Python library which is used to perform scientific computations.
- **Keras**
 - It is an open source library, which is used to work around the neural network.

Usage of Python

- Desktop Applications
- Web Applications
- [Data Science](#)
- [Artificial Intelligence](#)
- [Machine Learning](#)
- Scientific Computing
- [Robotics](#)
- [Internet of Things \(IoT\)](#)
- Gaming
- Mobile Apps
- Data Analysis and Preprocessing

Python Applications



How to Install Python (Environment Set-up)

- **<https://www.javatpoint.com/how-to-install-python>**

String

The String Data Type

Strings are one of the fundamental Python data types. The term **data type** refers to what kind of data a value represents. Strings are used to represent text.

We say that strings are a **fundamental** data type because they can't be broken down into smaller values of a different type. Not all data types are fundamental.

Strings have three properties that you'll explore in the coming sections:

1. Strings contain **characters**, which are individual letters or symbols.
2. Strings have a **length**, which is the number of characters contained in the string.
3. Characters in a string appear in a **sequence**, meaning each character has a numbered position in the string.

String Literals

As you've already seen, you can create a string by surrounding some text with quotation marks:

```
string1 = 'Hello, world'  
string2 = "1234"
```

Either single quotes (`string1`) or double quotes (`string2`) can be used to create a string, as long as both quotation marks are the same type.

Concatenation and indexing

```
>>> string1 = "abra"
>>> string2 = "cadabra"
>>> magic_string = string1 + string2
>>> magic_string
'abracadabra'
```

```
>>> first_name = "Arthur"
>>> last_name = "Dent"
>>> full_name = first_name + " " + last_name
>>> full_name
'Arthur Dent'
```

	f		i		g				p		i		e	
	0		1		2		3		4		5		6	

```
>>> flavor = "fig pie"
>>> flavor[1]
'i'
```

If you try to access an index beyond the end of a string, then Python raises an `IndexError`:

```
>>> flavor[9]
Traceback (most recent call last):
  File "<pyshell#4>", line 1, in <module>
    flavor[9]
IndexError: string index out of range
```

Negative indices

	f		i		g				p		i		e	
	-7		-6		-5		-4		-3		-2		-1	

```
>>> flavor[-1]
'e'
```

Just like with positive indices, Python raises an `IndexError` if you try to access a negative index less than the index of the first character in the string:

```
>>> flavor[-10]
Traceback (most recent call last):
  File "<pysHELL#5>", line 1, in <module>
    flavor[-10]
IndexError: string index out of range
```

String Slicing

Suppose you need a string containing just the first three letters of the string "fig pie". You could access each character by index and concatenate them like this:

```
>>> first_three_letters = flavor[0] + flavor[1] + flavor[2]
>>> first_three_letters
'fig'
```

Here's what this looks like for the string "fig pie":

	f		i		g				p		i		e	
0		1		2		3		4		5		6		7

```
>>> flavor = "fig pie"
>>> flavor[0:3]
'fig'
```

Strings Are Immutable

Strings are **immutable**, which means that you can't change them once you've created them. For instance, see what happens when you try to assign a new letter to one particular character of a string:

```
>>> word = "goal"
>>> word[0] = "f"
Traceback (most recent call last):
  File "<pyshell#16>", line 1, in <module>
    word[0] = "f"
TypeError: 'str' object does not support item assignment
```

If you want to alter a string, then you must create an entirely new string. To change the string "goal" to the string "foal", you can use a string slice to concatenate the letter "f" with everything but the first letter of the word "goal":

```
>>> word = "goal"
>>> word = "f" + word[1:]
>>> word
'foal'
```

Review Exercises

1. Create a string and print its length using `len()`.
2. Create two strings, concatenate them, and print the resulting string.
3. Create two strings, use concatenation to add a space between them, and print the result.
4. Print the string "zing" by using slice notation to specify the correct range of characters in the string "bazinga".

Manipulate Strings With Methods

Strings come bundled with special functions called **string methods** that you can use to work with and manipulate strings. There are numerous string methods available, but we'll focus on some of the most commonly used ones.

In this section, you'll learn how to:

- Convert a string to uppercase or lowercase
- Remove whitespace from a string
- Determine if a string begins or ends with certain characters

Converting String Case

To convert a string to all lowercase letters, you use the string's `.lower()` method. This is done by tacking `.lower()` onto the end of the string itself:

```
>>> "Jean-Luc Picard".lower()  
'jean-luc picard'
```

The dot (.) tells Python that what follows is the name of a method—the `lower()` method in this case.

```
>>> name = "Jean-Luc Picard"  
>>> name.lower()  
'jean-luc picard'
```

```
>>> name.upper()  
'JEAN-LUC PICARD'
```

```
>>> len(name)  
15
```

Without library function

- `str = input("Enter the String(Upper case):")`
- `i = 0`
- `ch2 = ""`
- `# convert capital letter string to small letter string`
- `while str[i]:`
 - `ch = ord(str[i])`
 - `if ch > 64 and ch < 91:`
 - `ch2 += chr(ch+32)`
 - `else:`
 - `ch2 += chr(ch)`
 - `i += 1`
- `print("Lower case String is:", ch2)`

Removing Whitespace From a String

There are three string methods that you can use to remove whitespace from a string:

1. `.rstrip()`
2. `.lstrip()`
3. `.strip()`

`.rstrip()` removes whitespace from the right side of a string:

```
>>> name = "Jean-Luc Picard      "  
>>> name  
'Jean-Luc Picard      '  
>>> name.rstrip()  
'Jean-Luc Picard'
```

`.lstrip()` works just like `.rstrip()`, except that it removes whitespace from the left-hand side of the string:

```
>>> name = "    Jean-Luc Picard"
>>> name
'    Jean-Luc Picard'
>>> name.lstrip()
'Jean-Luc Picard'
```

To remove whitespace from both the left and the right sides of the string at the same time, use `.strip()`:

```
>>> name = "    Jean-Luc Picard    "
>>> name
'    Jean-Luc Picard    '
>>> name.strip()
'Jean-Luc Picard'
```

Determine If a String Starts or Ends With a Particular String

```
>>> starship = "Enterprise"  
>>> starship.startswith("en")  
False
```

```
>>> starship.startswith("En")  
True
```

```
>>> starship.endswith("rise")  
True
```

```
>>> starship.endswith("risE")  
False
```

Review Exercises

1. Write a program that converts the following strings to lowercase:
"Animals", "Badger", "Honey Bee", "Honey Badger". Print each lowercase string on a separate line.
2. Repeat exercise 1, but convert each string to uppercase instead of lowercase.

3. Write a program that removes whitespace from the following strings, then print out the strings with the whitespace removed:

```
string1 = "    Filet Mignon"  
string2 = "Brisket    "  
string3 = "  Cheeseburger  "
```

4. Write a program that prints out the result of `.startswith("be")` on each of the following strings:

```
string1 = "Becomes"  
string2 = "becomes"  
string3 = "BEAR"  
string4 = "  bEautiful"
```

5. Using the same four strings from exercise 4, write a program that uses string methods to alter each string so that `.startswith("be")` returns `True` for all of them.

Interact With User Input

```
>>> input()
Hello there!
'Hello there!'
>>>
```

```
prompt = "Hey, what's up? "
user_input = input(prompt)
print("You said: " + user_input)
```

```
response = input("What should I shout? ")
shouted_response = response.upper()
print("Well, if you insist..." + shouted_response)
```

Review Exercises

1. Write a program that takes input from the user and displays that input back.
2. Write a program that takes input from the user and displays the input in lowercase.
3. Write a program that takes input from the user and displays the number of characters in the input

Working With Strings and Numbers

When you get user input using `input()`, the result is always a string. There are many other situations in which input is given to a program as a string. Sometimes those strings contain numbers that need to be fed into calculations.

Using Strings With Arithmetic Operators

```
>>> num = "2"  
>>> num + num  
'22'
```

```
>>> num = "12"  
>>> num * 3  
'121212'
```

```
>>> 3 * num  
'121212'
```

```
>>> "12" * "3"
```

Traceback (most recent call last):

File "<stdin>", line 1, in <module>

TypeError: can't multiply sequence by non-int of type 'str'

```
>>> "3" + 3
```

Traceback (most recent call last):

File "<stdin>", line 1, in <module>

TypeError: can only concatenate str (not "int") to str

Converting Strings to Numbers

The `TypeError` examples in the previous section highlight a common problem when applying user input to an operation that requires a number and not a string: type mismatches.

Let's look at an example. Save and run the following program:

```
num = input("Enter a number to be doubled: ")  
doubled_num = num * 2  
print(doubled_num)
```

If you entered the number 2 at the prompt, then you would expect the output to be 4. But in this case, you would get 22. Remember, `input()` always returns a string, so if you input 2, then `num` is assigned the string "2", not the integer 2. Therefore, the expression `num * 2` returns the string "2" concatenated with itself, which is "22".

To perform arithmetic on numbers contained in a string, you must first convert them from a string type to a number type. There are two functions that you can use to do this: `int()` and `float()`.

`int()` stands for **integer** and converts objects into whole numbers, whereas `float()` stands for **floating-point number** and converts objects into numbers with decimal points. Here's what using each one looks like in the interactive window:

```
>>> int("12")
```

```
12
```

```
>>> float("12")
```

```
12.0
```

Notice how float() adds a decimal point to the number. Floatingpoint numbers always have at least one decimal place of precision. For this reason, you can't change a string that looks like a floating-point number into an integer because you would lose everything after the decimal point .

Try converting the string "12.0" to an integer:

```
>>> int("12.0")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: invalid literal for int() with base 10: '12.0'
```

Even though the extra 0 after the decimal place doesn't add any value to the number, Python won't change 12.0 into 12 because it would result in a loss of precision.

Let's revisit the program from the beginning of this section and see how to fix it. Here's the code again:

```
num = input("Enter a number to be doubled: ")
doubled_num = num * 2
print(doubled_num)
```

The issue is on the line `doubled_num = num * 2` because `num` is a string and `2` is an integer.

You can fix the problem by passing `num` to either `int()` or `float()`. Since the prompts asks the user to input a number, and not specifically an integer, let's convert `num` to a floating-point number:

```
num = input("Enter a number to be doubled: ")  
doubled_num = float(num) * 2  
print(doubled_num)
```

Now when you run this program and input 2, you get 4.0 as expected. Try it out!

Converting Numbers to Strings

Since `num_pancakes` is a number, Python can't concatenate it with the string "I'm going to eat". To build the string, you need to convert `num_pancakes` to a string using `str()`:

```
>>> num_pancakes = 10
>>> "I am going to eat " + str(num_pancakes) + " pancakes."
'I am going to eat 10 pancakes.'
```

You can also call `str()` on a number literal:

```
>>> "I am going to eat " + str(10) + " pancakes."
'I am going to eat 10 pancakes.'
```

`str()` can even handle arithmetic expressions:

```
>>> total_pancakes = 10
>>> pancakes_eaten = 5
>>> "Only " + str(total_pancakes - pancakes_eaten) + " pancakes left."
'Only 5 pancakes left.'
```

Review Exercises

1. Create a string containing an integer, then convert that string into an actual integer object using `int()`. Test that your new object is a number by multiplying it by another number and displaying the result.
2. Repeat the previous exercise, but use a floating-point number and `float()`.
3. Create a string object and an integer object, then display them side by side with a single print statement using `str()`.
4. Write a program that uses `input()` twice to get two numbers from the user, multiplies the numbers together, and displays the result. If the user enters 2 and 4, then your program should print the following text:

The product of 2 and 4 is 8.0.

methods

1. String Comparison

Operators such as `==`, `<`, `>`, `<=`, `>=` and `!=` are used to compare the strings. Python compares strings by comparing their corresponding characters.

2. The `split()` Method

The `split()` method returns a list of all the words in a string. It is used to break up a string into smaller strings.

Example: Consider a input string that has a list of names of various multinational companies, such as TCS, INFOSYS, MICROSOFT, YAHOO and GOOGLE. Use split method and display the name of each company in a different line.

```
TOP_10_Company="TCS,INFOSYS,GOOGLE,MICROSOFT,YAHOO"  
Company=TOP_10_Company.split(",")  
print(Company)  
for c in Company:  
    print(end=" ")  
    print(c)
```

Output

```
['TCS', 'INFOSYS', 'GOOGLE', 'MICROSOFT', 'YAHOO']  
TCS  
INFOSYS  
GOOGLE  
MICROSOFT  
YAHOO
```

Testing String

- A string may contain digits, alphabets or a combination of both of these. Thus, various methods are available to test if the entered string is a digit or alphabet or is alphanumeric.

<i>Methods of str Class for Testing its Character</i>	<i>Meaning</i>
bool isalnum() Example: >>>S="Python Programming" >>>S.isalnum() False >>> S="Python" >>>S.isalnum() True >>> P="1Jhon" >>>P.isalnum() True	Returns True if characters in the string are alphanumeric and there is at least one character.

bool isalpha()

Example:

```
>>> S="Programming"
>>>S.isalpha()
True
>>> S="1Programming"
>>>S.isalpha()
False
```

Returns True if the characters in the string are alphabetic and there is at least one character.

bool isdigit()

Example:

```
>>> Str1="1234"
>>> Str1.isdigit()
True
>>> Str2="123Go"
>>> Str2.isdigit()
False
```

Returns True if the characters in the string contain only digits.

bool islower()

Example:

```
>>> S="hello"
>>>S.islower()
True
```

Returns True if all the characters in the string are in lowercase.

bool isupper()

Example:

```
>>> S="HELLO"
>>>S.isupper ()
True
```

Returns True if all the characters in the string are in uppercase.

bool isspace()

Example:

```
>>> S=" "
>>>S.isspace()
True
>>> Str1="Hello Welcome to Programming
World"
>>> Str1.isspace ()
False
```

Returns true if the string contains only white space characters.

Searching Substring in a String

<i>Methods of str Class for Searching the Substring in a Given String</i>	<i>Meaning</i>
bool endswith(str Str1) Example: >>> S="Python Programming" >>>S.endswith("Programming") True	Returns true if the string ends with the substring Str1.
bool startswith(str Str1) Example: >>> S="Python Programming" >>>S.startswith("Python") True	Returns true if the string starts with the substring Str1.
int find(str Str1) Example: >>> Str1="Python Programming" >>> Str1.find("Prog") 7#Returns the index from where the string "Prog" begins >>> Str1.find("Java") -1#Returns -1 if the string "Java" is not found in the string str1	Returns the lowest index where the string Str1 starts in this string or returns -1 if the string Str1 is not found in this string.

```
int rfind(str Str1)
```

Example:

```
>>> Str1="Python Programming"
```

```
>>> Str1.rfind("o")
```

9#Returns the index of last occurrence of string "o"
in Str1

Returns the highest index where the string Str1 starts in this string or returns -1 if the string Str1 is not found in this string.

```
int count(str S1)
```

Example:

```
>>> Str1="Good Morning"
```

```
>>> Str1.count("o")
```

```
3
```

Returns the number of occurrences of this substring.

Streamline Your Print Statements

Suppose you have a string, `name = "Zaphod"`, and two integers, `heads = 2` and `arms = 3`. You want to display them in the string `"Zaphod has 2 heads and 3 arms"`. This is called **string interpolation**, which is just a fancy way of saying that you want to insert some variables into specific locations in a string.

One way to do this is with string concatenation:

```
>>> name + " has " + str(heads) + " heads and " + str(arms) + " arms"  
'Zaphod has 2 heads and 3 arms'
```

This code isn't the prettiest, and keeping track of what goes inside or outside the quotes can be tough. Fortunately, there's another way of interpolating strings: **formatted string literals**, more commonly known as **f-strings**.

The easiest way to understand f-strings is to see them in action. Here's what the above string looks like when written as an f-string:

```
>>> f"{name} has {heads} heads and {arms} arms"  
'Zaphod has 2 heads and 3 arms'
```

There are two important things to notice about the above example:

1. The string literal starts with the letter `f` before the opening quotation mark.
2. Variable names surrounded by curly braces (`{}`) are replaced by their corresponding values without using `str()`.

You can also insert Python expressions between the curly braces. The expressions are replaced with their result in the string:

```
>>> n = 3  
>>> m = 4  
>>> f"{n} times {m} is {n*m}"  
'3 times 4 is 12'
```

It's a good idea to keep any expressions used in an f-string as simple as possible. Packing a bunch of complicated expressions into a string literal can result in code that is difficult to read and difficult to maintain.

f-strings are available only in Python version 3.6 and above. In earlier versions of Python, you can use `.format()` to get the same results. Returning to the Zaphod example, you can use `.format()` to format the string like this:

```
>>> "{} has {} heads and {} arms".format(name, heads, arms)
'Zaphod has 2 heads and 3 arms'
```


Review Exercises

1. Create a float object named `weight` with the value 0.2, and create a string object named `animal` with the value "newt". Then use these objects to print the following string using only string concatenation:

```
0.2 kg is the weight of the newt.
```

2. Display the same string by using `.format()` and empty `{}` placeholders.
3. Display the same string using an f-string.

Exercise

- Write the function `countB(word)` which takes a word as the argument and returns the number of 'b' in that word.
- Write the function `count_Letter(word, letter)` which takes a word and a letter as arguments and returns the number of occurrences of that letter in the word.
- Write the function `modify_Case(word)` which changes the case of all the letters in a word and returns the new word.
- A string contains a sequence of characters. Elements within a string can be accessed using an index which starts from 0. Write the function `getChar(word, pos)` which takes a word and a number as arguments and returns the character at that position.
- Write a program to read string and display "Total number of uppercase and lowercase letters".

Final Assignment

- Conversion of HexDecimal Number into its Equivalent Binary Number
- Implement number system conversion concept.
- Note: Use ord() to obtain the ASCII value of a character.

- **Sample Input**

Please Enter Hexadecimal Number: 12FD

Output

Equivalent Binary Number is

0001 0010 1111 1101

Numbers and Math

- **In this Section, you will learn how to:**
 - Work with Python's three built-in number types: integer, floating point, and complex numbers
 - Round numbers to a given number of decimal places
 - Format and display numbers in strings

Integers and Floating-Point Numbers

Python has three built-in number data types: integers, floating-point numbers, and complex numbers.

An integer is a whole number with no decimal places. For example, 1 is an integer, but 1.0 isn't. The name for the integer data type is int, which you can see with the type() function: You can create an integer by simply typing the number explicitly or using the int() function.

```
>>> type(1)
<class 'int'>
```

```
>>> int("25")
25
```

Integer Literal

An **integer literal** is an integer value that is written explicitly in your code, just like a string literal is a string that is written explicitly in your code. For example, `1` is an integer literal, but `int("1")` isn't.

Integer literals can be written in two different ways:

```
>>> 1000000
```

```
1000000
```

```
>>> 1_000_000
```

```
1000000
```

In Python, you can't use commas to group digits in integer literals, but you can use an underscore (`_`). The value `1_000_000` expresses one million in a more readable manner.

Floating-Point Numbers

A **floating-point number**, or **float** for short, is a number with a decimal place. 1.0 is a floating-point number, as is -2.75. The name of a floating-point data type is float:

```
>>> type(1.0)
<class 'float'>
```

Floats can be created by typing a number directly into your code, or by using the `float()` function. Like `int()`, `float()` can be used to convert a string containing a number to a floating-point number:

```
>>> float("1.25")
1.25
```


Floating Point Literal

A **floating-point literal** is a floating-point value that is written explicitly in your code. `1.25` is a floating-point literal, while `float("1.25")` is not.

Floating-point literals can be created in three different ways. Each of the following creates a floating-point literal with a value of one mil-

Example

```
>>> 1000000.0  
1000000.0
```

```
>>> 1_000_000.0  
1000000.0
```

```
>>> 1e6  
1000000.0
```

The first two ways are similar to the two methods for creating integer literals that you saw earlier. The second method, which uses underscores to separate digits into groups of three, is useful for creating float literals with lots of digits.

Exponential Notation

To write a float literal in E-notation, type a number followed by the letter `e` and then another number. Python takes the number to the left of the `e` and multiplies by 10 raised to the power of the number after the `e`. So `1e6` is equivalent to 1×10^6 .

Note

E-notation is short for **exponential notation**, and is the more common name for how many calculators and programming languages display large numbers.

Example

Python also uses E-notation to display large floating point numbers:

```
>>> 2000000000000000000.0  
2e+17
```

The float 2000000000000000000.0 gets displayed as 2e+17. The + sign indicates that the exponent 17 is a positive number. You can also use negative numbers as the exponent:

```
>>> 1e-4  
0.0001
```

The literal 1e-4 is interpreted as 10 raised to the power -4, which is 1/10000 or, equivalently, 0.0001.

Unlike integers, floats do have a maximum size. The maximum floating-point number depends on your system, but something like 2e400 ought to be well beyond most machines' capabilities. 2e400 is 2×10^{400} , which is far more than the **total number of atoms in the universe!**

When you reach the maximum floating-point number, Python returns a special float value `inf`:

```
>>> 2e400
inf
```

`inf` stands for infinity, and it just means that the number you've tried to create is beyond the maximum floating-point value allowed on your computer. The type of `inf` is still `float`:

```
>>> n = 2e400
>>> n
inf
>>> type(n)
<class 'float'>
```

There is also `-inf` which stands for negative infinity, and represents a negative floating-point number that is beyond the minimum floating-point number allowed on your computer:

```
>>> -2e400
-inf
```

Review Exercises

1. Write a script that creates the two variables, `num1` and `num2`. Both `num1` and `num2` should be assigned the integer literal 25,000,000, one written with underscored and one without. Print `num1` and `num2` on two separate lines.
2. Write a script that assigns the floating-point literal 175000.0 to the variable `num` using exponential notation, and then prints `num` in the interactive window.
3. In IDLE's interactive window, try and find the smallest exponent `N` so that `2e<N>`, where `<N>` is replaced with your number, returns `inf`.

Arithmetic Operators and Expressions

Addition

Addition is performed with the + operator:

```
>>> 1 + 2
3
>>> 1.0 + 2
3.0
```

Subtraction

To subtract two numbers, just put a - in between them:

```
>>> 1 - 1
0
>>> 5.0 - 3
2.0
```

Multiplication

To multiply two numbers, use the * operator:

```
>>> 3 * 3
9
>>> 2 * 8.0
16.0
```

```
>>> 1 - -3
4
>>> 1 --3
4
>>> 1- -3
4
>>>1--3
4
```

Division

The `/` operator is used to divide two numbers:

```
>>> 9 / 3
3.0
```

```
>>> 5.0 / 2
2.5
```

Integer Division

If writing `int(5.0 / 2)` seems a little long-winded to you, Python provides a second division operator, `//`, called the **integer division** operator:

```
>>> 9 // 3
3
```

```
>>> 5.0 // 2
2.0
```

Exponents

You can raise a number to a power using the `**` operator:

```
>>> 2 ** 2
4
```

```
>>> 2 ** 3
8
```

```
>>> 2 ** 4
16
```


Math Functions and NumberMethods

Python has a few built-in functions you can use to work with numbers. In this section, you'll learn about three of the most common ones:

1. `round()`, for rounding numbers to some number of decimal places
2. `abs()`, for getting the absolute value of a number
3. `pow()`, for raising a number to some power

Review Exercises

1. Write a script that asks the user to input a number and then displays that number rounded to two decimal places. When run, your program should look like this:

```
Enter a number: 5.432
5.432 rounded to 2 decimal places is 5.43
```

2. Write a script that asks the user to input a number and then displays the absolute value of that number. When run, your program should look like this:

```
Enter a number: -10
The absolute value of -10 is 10.0
```

3. Write a script that asks the user to input two numbers by using the `input()` function twice, then display whether or not the difference between those two number is an integer. When run, your program should look like this:

```
Enter a number: 1.5
Enter another number: .5
The difference between 1.5 and .5 is an integer? True!
```

If the user inputs two numbers whose difference is not integral, the output should look like this:

```
Enter a number: 1.5
Enter another number: 1.0
The difference between 1.5 and 1.0 is an integer? False!
```

Review Exercises

1. Print the result of the calculation $3 ** .125$ as a fixed-point number with three decimal places.
2. Print the number 150000 as currency, with the thousands grouped with commas. Currency should be displayed with two decimal places.
3. Print the result of $2 / 10$ as a percentage with no decimal places. The output should look like 20%.
4. Write a program to calculate the hypotenuse of the right-angled triangle

Complex Numbers

If you have ever taken a pre-calculus or higher level algebra math class, you may remember that a complex number is a number with two distinct components: a **real** component and an **imaginary** component.

There are several ways to denote complex numbers, but a common method is to indicate the real component with the letter i and the imaginary component with the letter j . For example, $1i + 2j$ is the complex number with real part 1 and imaginary part 2.

```
>>> a = 1 + 2j
```

```
>>> b = 3 - 4j
```

```
>>> a + b  
(4-2j)
```

```
>>> a - b  
(-2+6j)
```

```
>>> a * b  
(11+2j)
```

```
>>> a ** b  
(932.1391946432212+95.9465336603419j)
```

```
>>> a / b  
(-0.2+0.4j)
```

Interestingly, although not surprising from a mathematical point of view, `int` and `float` objects also have the `.real` and `.imag` properties, as well as the `.conjugate()` method:

```
>>> x = 42
>>> x.real
42
>>> x.imag
0
>>> x.conjugate()
42

>>> y = 3.14
>>> y.real
3.14
>>> y.imag
0.0
>>> y.conjugate()
3.14
```

For floats and integers, `.real` and `.conjugate()` always return the number itself, and `.imag` always returns 0. One thing to notice, however, is that `n.real` and `n.imag` return an integer if `n` is an integer, and a float if `n` is a float.

Now that you have seen the basics of complex numbers, you might be wondering when you would ever need to use them. If you are learning Python for web development or automation, the truth is you may *never* need to use complex numbers.

On the other hand, complex numbers are important in domains such as scientific computing and computer graphics. If you ever work in those domains, you may find Python's built-in support for complex **numbers useful.**

Types of Operator

There are seven types of operators supported by the Python language. They are:

1. Arithmetic operators
2. Relational or Comparison operators.
3. Assignment operators
4. Logical operators
5. Membership operators
6. Identity operators
7. Bitwise operators

Arithmetic Operators

```
x, y, z, f = 27, 10, -9, 4
```

```
a = x + y
```

```
print ("When you add x and y you get", a)
```

```
b = x - y
```

```
print ("When you subtract y from x you get", b)
```

```
c = x * y
```

```
print ("When you multiply x and y you get", c)
```

```
d = x / z
```

```
print ("When you divide x by z you get", d)
```

```
e = x % y
```

```
print (e, "is the remainder when x is divided by y")
```

```
g = x**f
```

```
print ("x to the power of 4 is", g)
```

```
h = x//y
```

```
print ("The floor division value of x divided by y is", h)
```

Operator	Description
+ Addition	Adds two values on either sides.
- Subtraction	Subtracts the value on the right side from that on the left.
* Multiplication	Multiplies values on either sides.
/ Division	Divides the value on the left side by that on the right.
% Modulus	Divides the value on the left by that on the right and returns the remainder.
** Exponent	Performs exponential (power) calculation of the value on the right by the value on the left.
// Floor Division	Divides the value on the left by that on the right and returns the quotient value of the result (without the decimal values).

Comparison Operators

```
x, y = 12, 10
```

```
if (x == y):
```

```
    print ("1. x is equal to y")
```

```
else:
```

```
    print ("1. x is not equal to y")
```

```
if (x != y):
```

```
    print ("2. x is not equal to y")
```

```
else:
```

```
    print ("2. x is equal to y")
```

```
if (x < y):
```

```
    print ("3. x is less than y")
```

```
else:
```

```
    print ("3. x is not less than y")
```

```
if (x > y):
```

```
    print ("4. x is greater than y")
```

```
else:
```

```
    print ("4. x is not greater than y")
```

```
if (x <= y):
```

```
    print ("5. x is either less than or equal to y")
```

```
else:
```

```
    print ("5. x is neither less than nor equal to y")
```

```
if (x >= y):
```

```
    print ("6. x is either greater than or equal to y")
```

```
else:
```

```
    print ("6. x is neither greater than nor equal to y")
```

Operator	Description
==	Condition returns true if the value of the left and right operands are equal.
!=	Condition returns true if the value of the left and right operands are not equal.
>	Condition returns true if the value of the left operand is greater than the value of the right operand.
<	Condition returns true if the value of the left operand is less than the value of the right operand.
>=	Condition returns true if the value of the left operand is greater than or equal to the value of the right operand.
<=	Condition returns true if the value of the left operand is less than or equal to the value of right operand.

Assignment Operators

```
x, y, z = 12, 7, 0
```

```
z = x + y
```

```
print ("1. Value of z is ", z)
```

```
z += x
```

```
print ("2. Value of z is ", z)
```

```
z *= x
```

```
print ("3. Value of z is ", z)
```

```
z /= x
```

```
print ("4. Value of z is ", z)
```

```
z = 2
```

```
z %= x
```

```
print ("5. Value of z is ", z)
```

```
z **= x
```

```
print ("6. Value of z is ", z)
```

```
z //= x
```

```
print ("7. Value of z is ", z)
```

Operator	Description
=	The equal signs assigns values on the right operand to that on the left side.
+= Add AND	Adds the value of the right operand to the value of the left operand then assign the result to left operand.
-= Subtract AND	Subtracts the value of the right operand from the value left operand then assigns the result to the left operand.
*= Multiply AND	Multiplies the value of the right operand with the value of the left operand then assigns the result to left operand.
/= Divide AND	Divides the value of the left operand by the value right operand then assigns the result to left operand.
%= Modulus AND	Takes the modulus using the values of the two operands then assigns the result to left operand.
**= Exponent AND	Performs exponential (power) calculation of the operators then assigns the result to the left operand.
//= Floor Division	It performs floor division on the operators then assigns the result to the left operand.

Logical Operators

```
x = True
y = False

print ("When x is True and y is False:")

print ("x and y is", x and y)

print("x or y is", x or y)

print("not x is", not x)

print("not y is", not y)
```

Operator	Meaning
and	Returns True if both the operands are True
or	Returns True if either of the operands is True
not	Returns True if operand is False and False if the operand is True.

Membership operators

```
x = "Hello World!"
```

```
y = {1:"x", 2:"y"}
```

```
print ("H" in x)
```

```
print ("Hello" not in x)
```

```
print (2 in y)
```

```
print ("z" in y)
```

Operator	Meaning
in	Returns True if the variable or variable is found in the sequence.
not in	Returns True if the variable or value is not found in the sequence.

Identity operators

```
x1 = 10
y1 = 10
x2 = "Hello"
y2 = "Hello"
x3 = [1, 2, 3]
y3 = [1, 2, 3]

print (x1 is not y1)
print (x2 is y2)
print (x3 is y3)
```

Operator	Meaning
is	Returns True if the operands are identical.
is not	Returns True if the operands are not identical.

Bitwise operators

Operator	Meaning
&	Bitwise AND
 	Bitwise OR
~	Bitwise NOT
^	Bitwise XOR
>>	Bitwise right shift
<<	Bitwise left shift

The Bitwise AND (&) Operator

- This operator performs AND operation on input bits of numbers. The Bitwise AND operator is represented as '&'. The '&' operator operates on two operands bit-by-bit.

<i>Input</i>		<i>Output</i>
X	Y	X & Y
0	0	0
0	1	0
1	0	0
1	1	1

We can conclude from this table that the output is obtained by multiplying the input bits.

Example of AND Operator

```
>>> 1 & 3
```

```
1      # The bitwise & operator on 1 and 3 returns 1
```

```
>>> 5 & 4
```

```
4      # The bitwise & operator on 5 and 4 returns 4
```

1 and 3 are converted into their equivalent binary format					
	0	0	0	1	(one)
&					
	0	0	1	1	(Three)

Bitwise operation	(0 & 0)	(0 & 0)	(0 & 1)	(1 & 1)	

Result	0	0	0	1	(One)
Decimal equivalent of (0 0 0 1) = 1					
Therefore, 1 & 3 = 1					

- Write a program to read two numbers from the user. Display the result using bitwise & operator on the numbers.

```
num1 = int(input('Enter First Number: '))
num2 = int(input('Enter Second Number: '))
print(num1, ' & ', num2, ' = ', num1 & num2)
```

Output

```
#Test Case 1
Enter First Number: 1
Enter Second Number: 3
1 & 3 = 1
```

```
#Test Case 2
Enter First Number: 5
Enter Second Number: 6
5 & 6 = 4
```

The Bitwise OR (|) Operator

This operator performs bitwise OR operation on the numbers. The bitwise OR operator is represented as '|'. It also operates on two operands and the two operands are compared bit-by-bit.

<i>Input</i>		<i>Output</i>
X	Y	X Y
0	0	0
0	1	1
1	0	1
1	1	1

We can conclude from this table that the output is obtained by adding the input bits.

Examples of Bitwise '|' (OR) Operator

```
>>> 3 | 5
```

```
7      # The bitwise | operator on 3 and 5 returns 7
```

```
>>> 1 | 5
```

```
5      # The bitwise | operator on 1 and 5 returns 5
```

Working of the bitwise OR ('|') operator is given as follows:

Working of expression $3 \mid 5$ is as below.

Initially 3 and 5 are converted into their equivalent binary format

	0	0	1	1	(Three)
	0	1	0	1	(Five)

Bitwise operation (0 | 0) (0 | 1) (1 | 0) (1 | 1)

Result 0 1 1 1 (seven)

Decimal Equivalent of (0 1 1 1) = 7

Therefore $3 \mid 5 = 7$

Write a program to read two numbers from the user. Display the result using bitwise | operator on the numbers.

```
num1 = int(input('Enter First Number: '))
num2 = int(input('Enter Second Number: '))
print(num1, ' | ', num2, ' = ', num1 | num2)
```

Output

```
#Test Case 1
Enter First Number: 3
Enter Second Number: 5
3 | 5 = 7
```

```
#Test Case 2
Enter First Number: 6
Enter Second Number: 1
6 | 1 = 7
```

The Bitwise XOR (^) Operator

This operator performs bitwise exclusive or XOR operation on the numbers. It is represented as '^'. The '^' operator also operates on two operands and these two operands are compared bit-by-bit.

<i>Input</i>		<i>Output</i>
X	Y	$X \wedge Y$
0	0	0
0	1	1
1	0	1
1	1	0

We can conclude from this table that the output is logic one when one of the input bits is logic one.

Examples of Bitwise XOR (^) Operator

```
>>> 3 ^ 5
```

```
6      # The bitwise ^ operator on 3 and 5 returns 6
```

```
>>> 1 ^ 5
```

```
4      # The bitwise ^ operator on 1 and 5 returns 4
```

Working of the bitwise XOR (^) operator is given as follows:

Working of expression $3 \wedge 5$ is as below.

Initially 3 and 5 are converted into their equivalent binary format

0 0 1 1 (Three)

^

0 1 0 1 (Five)

Bitwise operation (0 ^ 0) (0 ^ 1) (1 ^ 0) (1 ^ 1)

Result 0 1 1 0 (Six)

Decimal Equivalent of (0 1 1 0) = 6

Therefore $3 \wedge 5 = 6$

Write a program to read two numbers from the user. Operate bitwise ^ operator on them and display the result.

```
num1 = int(input('Enter First Number: '))
num2 = int(input('Enter Second Number: '))
print(num1, ' ^ ', num2, ' = ', num1 ^ num2)
```

Output

#Test Case 1

Enter First Number: 3

Enter Second Number: 5

3 ^ 5 = 6

#Test Case 2

Enter First Number: 1

Enter Second Number: 2

1 ^ 2 = 3

The Right Shift (>>) Operator

The right shift operator is represented as >>. It also needs two operands. It is used to shift bits to the right by n position. Working of the right shift operator (>>) is explained as follows:

Example

```
>>>4 >> 2 # The input data 4 is to be shifted by 2 bits towards the right side
1
>>>8>>2
2
```

Consider the expression $4 \gg 2$.

Initially, the number 4 is converted into its corresponding binary format, i.e. 0 1 0 0

0 0 0 0 0 0 1 0 0 Binary 4 ←

8 7 6 5 4 3 2 1 0 Bit Index ←

The input data 4 is to be shifted by 2 bits towards the right side.

The answer in binary bits would be

0 0 0 0 0 0 0 0 1 Binary 1 ←

8 7 6 5 4 3 2 1 0 Bit Index ←



Note: Shifting the input number by N bits towards the right means the number is divided by 2^S . In short, it means $Y = N/2^S$.

Where,

N = The Number

S = The Number of Bit Positions to Shift

Consider the above example $4 \gg 2$. Let us solve this using the above formula, i.e. $y = N / 2^S$

$$= 4 / 2^2$$

$$= 4 / 4$$

$$= 1$$

Therefore, $4 \gg 2$ returns 1 in Python interactive mode.

Write a program to shift input data by 2 bits towards the right.

```
N = int(input('Enter  Number: '))
S = int(input('Enter Number of Bits to be shift Right: '))
print(N,' >> ',S,' = ', N >> S)
```

Output

```
Enter  Number: 8
Enter Number of Bits to be shift Right: 2
8  >>  2  =  2
```

The Left Shift (<<) Operator

The left shift operator is represented as <<. It also needs two operands. It is used to shift bits to the left by N position. The working of the left shift operator is given as follows:

Example

```
>>> 4 << 2 # The input data 4 is to be shifted by 2 bits towards the left side
16
```

```
>>> 8 << 2 # The input data 8 is to be shifted by 2 bits towards the left side
32
```

Explanation

Consider the expression `4 << 2`.

Initially, the number 4 is converted into its corresponding binary format, i.e. 0 1 0 0

0 0 0 0 0 0 1 0 0

Binary 4 ←

8 7 6 5 4 3 2 1 0

Bit Index ←

The input data 4 is to be shifted by 2 bits towards the left side.

The answer in binary bits would be

0 0 0 0 1 0 0 0 0

Binary 16 ←

8 7 6 5 4 3 2 1 0

Bit Index ←



Note: Shifting the input number by N bits towards the left side means the number is multiplied by 2^S . In short, it means $Y = N * 2^S$.

Where,

N = The Number

S = The Number of Bit Positions to Shift

Consider the above example $4 \ll 2$. Let us solve this using the above formula, i.e.

$$\begin{aligned} y &= N * 2^S \\ &= 4 * 2^2 \\ &= 4 * 4 \\ &= 16 \end{aligned}$$

Therefore, $4 \ll 2$ returns 16 in Python interactive mode.

Write a program to shift input data by four bits towards the left

```
N = int(input('Enter Number: '))
S = int(input('Enter Number of Bits to be shift Left: '))
print(N, ' << ', S, ' = ', N << S)
```

Output

```
Enter Number: 4
Enter Number of Bits to be shift Left: 2
4 << 2 = 16
```

Write a program using compound assignment operators to calculate the area of a circle.

```
radius = eval(input('Enter the Radius of Circle: ')) #Read Radius
print(' Radius = ',radius) #Display Radius
area = 3.14
radius **=2      #Radius = Radius ** 2
area*=radius     #Area=Area*Radius
print(' Radius of Circle is  = ',area)  #Print area
```

Output

```
Enter the Radius of Circle: 2
Radius =  2
Radius of Circle is  =  12.56
```

Goods Service Tax (GST) Calculator

What is GST?

Goods and services tax is a comprehensive tax levied on the manufacture, sale and consumption of goods and services at a national level. This tax has substituted all indirect taxes levied on goods and services earlier by the central and state governments in India.

Problem Statement

We all buy various goods from a store. Along with the price of the goods we wish to buy, we also have to pay an additional tax, which is calculated as a specific percentage on the total price of the goods. This is called GST on the products.

Model of GST Using an Example

The GST has two components, viz. one which is levied by the central government (referred to as central GST or **CGST**), and one levied by the state government (referred to as state GST or **SGST**). The **rates** for central GST and state GST are given as follows:

<i>Type of Tax</i>	<i>Tax Rate</i>
CGST	@9%
SGST	@9%

Example

Invoice of a product

<i>Particulars</i>	<i>GST on Particulars</i>
Cost of Production	5,000
Add: CGST @ 9%	450
Add: SGST @ 9%	450
Total Cost of Product:	₹5,900

Formula to Calculate Total Cost

(CGST Tax Rate on product) + (SGST Tax Rate on product)

Operator Precedence in Python

	Operator	Description
1	()	Parentheses
2	**	Exponentiation (raise to the power)
3	~ + -	Complement, unary plus +@ and minus -@
4	* / % //	Multiply, divide, modulo and floor division
5	+ -	Addition and subtraction
6	>> <<	Right and left bitwise shift
7	&	Bitwise 'AND'
8	^	Bitwise exclusive 'OR' and regular 'OR'
9	<= < > >=	Comparison operators
10	<= == !=	Equality operators
11	= %= /= //= -= += *= **=	Assignment operators
12	Is, is not	Identity operators
13	In, not in	Membership operators
14	not, or, and	Logical operators

Thank You